

Applicant: Kofi Sampong
Program: B.Eng., Computer Engineering
Email: ksampong@uoguelph.ca
Date: September 4, 2025

Reader's Note

These essays accompany my research proposal. They are not meant to replace the formal document; rather, they provide a narrative articulation of the **background**, the **core argument**, and a **proposed game plan**. If the standard proposal format feels compressed, skimming these pieces should leave a clearer picture of what I'm proposing, why it matters, and how I intend to proceed.

Suggested order: **Essay 1 (Introduction)** → **Essay 2 (Background/Context)** → **Essay 3 (Proposed Methodolgy)**. Each essay can also be read independently.

Introduction

Essay 1 - Rethinking the Black-Box Paradigm: Toward Structured Representations for Mechanistic Reasoning in Language Models

The current trajectory of machine learning emphasizes scaling models with more data and parameters to achieve better performance. While this paradigm has yielded powerful results, particularly in the realm of large language models (LLMs), it also presents significant limitations. These models remain fundamentally black boxes—producing outputs that are difficult to interpret, trust, or mechanistically understand. Drawing from interpretability literature, especially the work of Christoph Molnar, this paper critiques the over-reliance on post-hoc methods and unstructured representations. We argue that if the goal is to enable true reasoning—particularly causal, mechanistic, and goal-directed reasoning—then a shift toward structured, interpretable encodings of knowledge is necessary.

1. Introduction: Success and the Illusion of Understanding

The rise of large language models (LLMs) has brought with it remarkable capabilities across tasks such as translation, summarization, and dialogue. Their success has validated the long-debated proposition that natural language can serve as a powerful representation of knowledge, as noted by scholars such as Christopher Manning. However, this success has also contributed to a dangerous illusion: that these systems understand, reason, or "think" in any meaningful way. In reality, they remain fundamentally probabilistic sequence predictors—linguistic engines trained to generate coherent continuations based on prior context.

This disconnect between perceived and actual capabilities is exacerbated by the **black-box nature** of modern ML models. We often lack clarity on how decisions are made, what representations are learned, or whether these representations generalize safely across contexts.

2. Pitfalls of the Black-Box Approach

Christoph Molnar's work on model interpretability outlines a number of systemic pitfalls that continue to plague machine learning development:

- **Assuming generalization:** Models are often interpreted under the assumption that they generalize well. Yet if a model is underfit or overfit, any interpretation of its behavior is likely to be misleading.
- **Unnecessary complexity:** In many applications, simple, interpretable models would suffice—yet complex black-box models are still used, obfuscating the reasoning process.
- **Ignoring feature dependencies:** Interpretability tools often assume feature independence, which is rarely true in real-world data.
- **Confusing correlation for causation:** This results in overconfidence in what the model has "learned."

These pitfalls highlight that even **post-hoc interpretability methods**, which aim to extract insights from opaque models, are often susceptible to the same structural flaws embedded in the training process.

3. On the Limits of Language Models

At their core, LLMs are language machines. They do not reason causally, nor do they possess goal-directed cognition. Instead, they rely on the statistical regularities of language, generating outputs that *appear* meaningful because they are well-formed and contextually coherent.

But coherence is not causality. These models lack the ability to engage in **robust, step-by-step reasoning**—a hallmark of human cognition. They cannot independently construct and trace a causal chain of events, nor can they consistently represent concepts in a manner that reflects the underlying structure of the world.

4. Emergence vs. Attribution

There is growing concern around the **over-attribution of intelligence** to models based on their outputs. Emergent behavior is often cited as evidence that complex reasoning capabilities arise organically from scale. However, this view fails to distinguish between **emergent statistical behavior** and **structured, intentional reasoning**.

Rather than interpret emergent behavior as a sign of intelligence, we should recognize it as a byproduct of complex correlations—correlations that are not inherently causal or grounded in mechanistic understanding.

5. The Case for Structured Representations

If our aim is to enable **models that can reason, not just recall**, we must consider new approaches to how knowledge is encoded and used. While natural language has proven to be a surprisingly effective medium for modeling certain kinds of knowledge, it remains insufficient for tasks that require causal understanding, planning, and mechanistic inference.

We propose that model architectures—and the training objectives that guide them—must begin to incorporate **formal, structured representations** of knowledge and tasks. These representations should explicitly encode:

- **Causal relationships** between concepts
- **Goals and subgoals** in multi-step reasoning processes
- **Procedural logic** required to solve tasks in a verifiable, interpretable manner

Such structure can serve as **scaffolding** for more interpretable, controllable, and trustworthy models.

6. Conclusion

LLMs have shown that text can encode much more than previously believed. However, they have also revealed the limits of unstructured representations and post-hoc interpretability. If we continue to treat black-box systems as if they inherently possess human-like reasoning capabilities, we risk building powerful tools we cannot fully understand or trust.

The next generation of intelligent systems must go beyond linguistic pattern recognition. They must **reason through** knowledge, not simply **reproduce** it. Achieving this will require a departure from scale-as-solution thinking, and a move toward structured, causal, and mechanistic frameworks for knowledge representation and inference.

References

- Molnar, C. (2020). *Interpretable Machine Learning*. Leanpub.
 - Manning, C. D. (2019). *Speech at Stanford on the Future of NLP and Representation*.
 - Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*.
-

Context

Essay 2 -A Case Study: When Fine-Tuning Fails to Deliver Meaning

I became acutely aware of these issues during my time as a research assistant on a deep learning project focused on educational tooling for undergraduate C/C++ instruction. The project involved designing and testing three model pipelines: one for **code summarization**, another for **code similarity**, and a third for **code correctness classification**. Each was meant to support teaching and assessment by automatically analyzing student code submissions.

While the summarization and similarity pipelines produced expected results, the correctness pipeline exposed serious limitations. This component aimed to classify student submissions as correct or incorrect using fine-tuned versions of CodeT5 and CodeBERT on labeled samples from the **Google Code Jam (GCJ)** and **CodeNet** datasets. Despite employing a sizable and well-labeled dataset, the pipeline struggled to produce meaningful distinctions between correct and incorrect code. Performance plateaued quickly, and none of the metrics (accuracy, precision, F1 score) suggested any genuine understanding of *why* a code sample was correct—only that it looked statistically similar to other samples.

Upon deeper investigation, detailed in our paper, we confirmed our initial suspicion: these models, even when fine-tuned, were not capturing the **functional meaning** of the code. Rather, they were relying on shallow linguistic and syntactic patterns—effectively recognizing surface-level statistical regularities rather than reasoning through the logic of the program. Their high performance on simpler problems (e.g., Type-II clones) gave the illusion of success, but they quickly broke down on more complex or semantically diverse examples (e.g., Type-IV problems), where **true understanding of program behavior** was required.

This experience illustrated two key points. First, it confirmed the **overconfidence** often placed in fine-tuning black-box models to solve specialized tasks—especially those requiring deeper logical reasoning. Second, it highlighted the **inability of language models to think mechanistically, logically, or causally**, especially when the evaluation involves nontrivial code behavior. What they "understand" is best interpreted as **approximate linguistic fluency**, not computational intent or correctness.

References

- Allister Pais, N., et al. (2025). *Evaluating Source Code Embeddings for Educational Downstream Tasks*. IEEE UEMCON 2025, status: under review.

Proposed Methodology

Essay-3 Toward Structured Representations for Mechanistic Understanding

In the broader context of mechanistic reasoning, causal modeling, interpretability, and explainable AI, I believe our specific case—modeling code correctness in educational tasks—offers a uniquely tractable environment for exploring models with such capabilities. Unlike abstract domains where concepts are often difficult to quantify or structurally represent, programming languages provide a well-defined, syntactically constrained, and semantically grounded instruction set. This makes them an ideal testbed for developing models that move beyond surface-level statistical correlations and begin to exhibit structured, functional understanding.

Rather than treating source code as mere text, we propose shifting toward **structured, task-informed representations** that explicitly encode **program flow**, **logical order**, and **functional transformations**. By giving models a more formal and interpretable view of what happens during execution—what precedes what, what affects what—we hope to encourage the formation of **latent spaces** that capture not just how the code *looks*, but what it *does* to its input over time.

Concept Bottlenecks as Inspiration

One direction I would like to explore draws inspiration from **concept bottleneck models**, which have been applied in the vision domain to improve interpretability and control. These models operate by passing data through a constrained intermediate layer (the "bottleneck") composed of **interpretable, labeled concepts** before producing the final output. In our case, we envision training a contrastive embedding head to project code into a latent space organized around **structural or functional representations** rather than purely linguistic ones.

The goal is to force the model to learn *which behaviors* or *logical structures* are shared across functionally similar programs, even when their syntax varies widely. In doing so, we move toward an embedding space where **semantic similarity** and **functional correctness** are more faithfully encoded.

Graph-Based Representations of Code

A promising representation to support this approach is **graph-based modeling**. Graphs naturally encode both **precedence** (which action occurs before another) and **composition** (how complex behaviors are built from simpler ones). By modeling programs as graphs—where nodes represent actions or operations and edges represent control or data flow—we can explicitly represent causality and hierarchy within a program's structure.

If we define a **closed set of fundamental operations**—akin to a minimal instruction set or "vocabulary" of base-level programming actions—then any higher-level behavior could be seen as a **composition of subgraphs**. In this framework, each code snippet could be embedded as a structured graph, and its representation would emerge as a function of the **sub-actions** it performs and the **transformations** it applies to its inputs.

This idea has natural parallels in **hardware compilation**, such as **FPGA optimizations**, where code is often translated into **dataflow graphs** to reason about computation and parallelism. Similarly, we could explore mappings from source code to **ASTs**, **dataflow diagrams**, or even low-level **assembly representations**—all of which offer different lenses into the code's functionality.

Ideal Outcome

The ideal outcome of this approach would be a system where code is no longer just a sequence of tokens, but a **structured object composed of interpretable primitives**, where:

- Every function can be broken down into a small set of **base actions**;
- These base actions act as **features** in the embedding space;
- The **latent space itself** reflects transformations applied to data;
- And similarity in the space reflects **functional equivalence** or shared procedural logic.

This, we hope, will push us closer to models that do not merely *describe* or *summarize* code but begin to **understand and reason through it**, forming internal representations that are **causal**, **logical**, and **mechanistic**.